Rodrigo Fernandez

Senior Go Backend Engineer
Katy, Texas | +1 (430) 279-2231| https://www.linkedin.com/in/rodrigo-bermejo-fern%C3%A1ndez-9a5a0664/| rodrigobfdev@gmail.com

Summary

Senior **Go Backend Engineer** with strong experience in fintech, healthcare, SaaS, and cloud infrastructure, building high-throughput APIs, event-driven services, and secure distributed platforms using **Go 1.17–1.22**, **gRPC**, **REST**, **PostgreSQL 12–16**, **Redis 6–7**, **Kafka 2.8–3.x**, **Docker 20–24**, **Kubernetes 1.22–1.29**, and **AWS**. Skilled in modernizing legacy services, fixing production latency, race-condition, and deployment issues, improving observability, and delivering reliable microservices with clean architecture, CI/CD, and measurable business impact across regulated customer systems. Comfortable leading code reviews, rollout planning, and production support.

Skills

- **Languages: Go 1.17–1.22**, SQL, Bash, Python 3.8–3.11, TypeScript 4.x–5.x
- **Backend: gRPC**, REST APIs, GraphQL basics, Gin, Echo, Chi, Go Kit, Clean Architecture, Hexagonal Architecture, Domain-Driven Design
- **Cloud & DevOps: AWS**, Lambda, ECS, EKS, S3, SQS, SNS, CloudWatch, IAM, Docker 20–24, Kubernetes 1.22–1.29, Helm 3.x
- **Databases: PostgreSQL 12–16**, MySQL 8.x, Redis 6–7, DynamoDB, MongoDB 5–6, database migration, indexing, query tuning
- **Messaging & Streaming: Kafka 2.8–3.x**, RabbitMQ 3.x, NATS, event-driven architecture, retry queues, dead-letter queues
- **Testing & Quality:** Go testing package, Testify, Mockery, integration testing, contract testing, load testing, race detection, static analysis
- **Observability:** Prometheus, Grafana, OpenTelemetry, CloudWatch, structured logging, distributed tracing, SLOs, alert tuning
- **CI/CD:** GitHub Actions, GitLab CI, Jenkins, Docker build pipelines, blue-green deployment, canary rollout, rollback strategy
- **Security:** OAuth2, JWT, RBAC, IAM least privilege, secrets management, API rate limiting, secure service-to-service communication

Experience

Lightedge — Senior Software Engineer *(Apr 2020 – Mar 2026)*

- Led backend modernization for a cloud infrastructure platform by migrating critical services from mixed legacy APIs to **Go 1.17–1.22** microservices with clearer ownership, cleaner boundaries, and safer deployment paths.
- Designed high-throughput **REST** and **gRPC** services using **Go**, PostgreSQL, Redis, and Kafka, reducing average API latency by **38%** while improving service reliability during peak customer traffic.
- Resolved a production race-condition issue in a billing synchronization worker by using Go race detection, context cancellation, idempotency keys, and safer transaction handling across retry flows.
- Migrated container workloads from manually managed Docker deployments to **Kubernetes 1.22–1.29** with Helm charts, health probes, resource limits, and rollback-ready release strategies.

- Implemented event-driven provisioning workflows with **Kafka 2.8–3.x**, retry topics, and dead-letter queues, reducing failed infrastructure provisioning cases across customer onboarding flows.
- Improved PostgreSQL performance by rewriting slow queries, adding partial indexes, and optimizing connection pooling, which reduced report generation time by **44%** for large enterprise tenants.
- Built secure service-to-service communication with JWT validation, scoped internal tokens, and **AWS IAM** policies, strengthening access control without slowing developer delivery.
- Introduced OpenTelemetry tracing, Prometheus metrics, and Grafana dashboards for Go services, helping teams identify memory pressure, queue lag, and downstream timeout issues faster.
- Refactored monolithic background jobs into smaller Go workers with context-aware cancellation, structured logging, and graceful shutdown behavior for safer deployments.
- Created CI/CD pipelines with GitHub Actions, Docker multi-stage builds, automated tests, and vulnerability scans, reducing release preparation effort.
- Supported production incidents by analyzing logs, traces, metrics, and database locks, then converted recurring failure patterns into preventive alerts and engineering tasks.
- Mentored engineers on idiomatic **Go**, interface design, error wrapping, concurrency patterns, and testing strategy so new services stayed maintainable as the platform expanded.

NIX United —Software Engineer *(Oct 2016 – Mar 2020)*

- Developed backend services for fintech and healthcare clients using **Go 1.11–1.16**, PostgreSQL, Redis, RabbitMQ, and Docker, with strong focus on API reliability and maintainable service design.
- Migrated legacy PHP and Node.js backend modules into Go services where performance, concurrency, and simpler deployment artifacts were important for client-facing workloads.
- Implemented payment reconciliation workflows with Go workers, RabbitMQ queues, retry policies, and database-level idempotency, reducing duplicate transaction defects.
- Fixed a difficult memory growth issue in a long-running Go service by profiling heap allocations, reducing unnecessary object retention, and improving batch processing logic.
- Built secure REST APIs with middleware for authentication, request validation, rate limiting, structured errors, and audit logging across regulated business workflows.
- Improved PostgreSQL query performance by normalizing overloaded tables, tuning indexes, and separating read-heavy reporting queries from transactional paths.
- Created Docker-based local development environments that helped engineers reproduce production-like issues faster and reduced onboarding setup time by **40%**.
- Added integration tests around critical billing and account-management paths using Go test suites, mocks, test containers, and seeded database fixtures.
- Implemented background notification services with retry handling, timeout control, and message deduplication to avoid repeated customer alerts during provider instability.
- Collaborated with frontend and QA teams to clarify API contracts, reduce breaking changes, and deliver backend features that matched real product behavior.
- Supported release stabilization by reviewing logs, fixing flaky integration tests, and improving rollback instructions before client production deployments.

Webiz Digital — Software Developer *(Jan 2014 – Sep 2016)*

- Built web platform backend modules using **Go 1.6–1.10**, REST APIs, MySQL, PostgreSQL, Redis, and Docker for customer portals, reporting tools, and content-heavy business applications.
- Converted selected slow API endpoints from legacy scripting services into Go handlers, improving response time by **33%** for traffic-heavy dashboard pages.
- Implemented authentication and session flows with secure cookies, JWT-based API access, password hashing, and role-based permissions for internal admin systems.

- Solved intermittent timeout errors by adding request context deadlines, database connection limits, and clearer upstream error handling across service calls.
- Created reusable Go packages for validation, logging, pagination, and API response formatting, helping developers ship consistent backend features faster.
- Improved Redis caching for frequently requested content and lookup data, reducing repeated database reads during normal customer activity.
- Integrated third-party payment, email, and analytics APIs with defensive retry logic, structured errors, and detailed audit logs for easier support troubleshooting.
- Added database migration scripts and safer deployment notes so schema changes could be released without blocking active application usage.
- Worked closely with product stakeholders to translate vague business requests into practical API contracts, acceptance scenarios, and incremental delivery plans.
- Reviewed pull requests for Go code quality, SQL safety, error handling, and test coverage so backend changes stayed stable across multiple client projects.

Aurio — Junior Software Developer *(Oct 2012 – Dec 2013)*

- Supported backend development for internal business applications using early **Go 1.2–1.5**, SQL, Linux, and REST-style services under senior engineering guidance.
- Assisted with building lightweight Go utilities for data import, log parsing, and scheduled processing, reducing repetitive manual support work by **22%**.
- Helped troubleshoot API bugs involving invalid payload formats, missing database records, and unclear error responses by improving validation and logging.
- Created SQL queries, stored procedures, and small service endpoints for reporting features used by operations and customer support teams.
- Worked on Docker-based experiments for local service packaging as the engineering team started standardizing development environments.
- Improved test coverage around utility packages, input parsing, and database access helpers so small changes were easier to review and maintain.
- Documented setup steps, common errors, and local troubleshooting commands to help new developers run backend services with fewer blockers.
- Collaborated with senior engineers on code reviews, debugging sessions, and production support tasks while learning practical Go service design.
- Contributed to small performance improvements by simplifying loops, reducing unnecessary database calls, and caching repeated lookup values.

Microsoft — Software Developer Intern *(Jun 2012 – Sep 2012)*

- Supported internal engineering tools using **C#**, **.NET Framework 4.0–4.5**, **ASP.NET MVC 3–4**, JavaScript, jQuery, and SQL Server, gaining practical experience in enterprise software quality and structured development workflows.
- Assisted senior engineers with debugging UI defects, validation issues, and data-handling problems, documenting root causes clearly so fixes could be reviewed and released with lower regression risk.
- Built small internal web modules for tracking records, reviewing status changes, and improving team visibility, applying clean HTML, CSS, JavaScript, and backend integration patterns.
- Resolved early-stage bugs related to inconsistent form validation and SQL query results by reproducing issues locally, checking logs, and working with mentors to apply safer input-handling rules.
- Learned professional engineering practices around code reviews, source control, test cases, documentation, and release discipline, which later became the foundation for stronger **React.js** and full-stack delivery.

Education

Texas State University
Bachelor's Degree in Computer Science (*Sep 2008 – May 2012)*